

GITWRATH: A Proof-Carrying, Agent-First Git Control Plane

GITWRATH Project

Draft for V3.01

Agent-first GitLab control plane, VTI, evidence, and merge-gate architecture

Abstract—Modern software agents can edit code faster than conventional review, CI, and release systems can safely absorb. GitHub Copilot cloud agent can research, plan, edit branches, run tests, and create pull requests, but public documentation describes important constraints: one branch at a time, one pull request per task, and GitHub-hosted repositories only [1]. GitLab Duo Agent Platform brings agentic automation into GitLab contexts, including issues, merge requests, pipelines, planning, and security signals [2]–[4]. Open-source agents such as OpenHands, SWE-agent, and Aider improve code authoring and task execution, but generally remain outside the Git/CI authority boundary [5]–[7]. GITWRATH proposes a different layer: an agent-first delivery control plane that treats agent intent, validation plans, CI execution, runner trust, cache use, failure evidence, merge gates, and release gates as one proof loop. Its VTI subsystem maps changes to validation with conservative fallback, while the V3.01 plan upgrades this into proof receipts, selector-miss learning, typed capabilities, admission enforcement, and a TUI operations console.

Index Terms—software engineering agents, CI/CD, GitLab, smart test selection, validation, evidence, merge gates, agent safety

I. INTRODUCTION

Agent-first development changes the bottleneck. Code generation is no longer the only scarce resource; trustworthy validation becomes scarce. Public coding-agent workflows now let developers delegate issue implementation, branch creation, test execution, and pull request creation [1], [8]. GitLab has similarly moved toward lifecycle-wide agentic automation across project context, CI/CD, security findings, and planning surfaces [2], [3], [9]. Open-source agents have also raised expectations for terminal use, repository exploration, and autonomous repair [5]–[7]. These systems are important, but they leave a gap for teams that need local-first, policy-controlled, proof-scoped autonomy over runners, sandboxes, cache, merge gates, release gates, and evidence retention.

GITWRATH is designed around that gap. Instead of making an agent imitate a human in a browser, it gives agents typed control surfaces: capability intents, validation planning, job traces, failure capsules, merge readiness, release status, cache verdicts, runner pools, and a TUI that exposes the same state to human supervisors. The long-term claim is not that agents should receive unrestricted control. It is that agents should receive bounded authority and produce durable evidence for every action that matters.

This paper describes the V3.01 direction for GITWRATH. It uses the repository itself as the artifact under study: a

Rust workspace with a `vgit` binary, proof-scoped support crates, GitLab-oriented orchestration, VTI smart test selection, a capability API, a Ratatui console, and deterministic paper screenshots. The contribution is a control-plane pattern for agent-first development:

- 1) a proof loop that connects intent, policy, validation, evidence, and merge or release decisions;
- 2) a VTI model that saves test time only when it can explain what was skipped and why;
- 3) a capability model that makes action availability canonical across API, CLI, and TUI surfaces;
- 4) a screenshot-capable operations console that makes agent behavior reviewable; and
- 5) a V3.01 roadmap for hardening admission, state, merge gates, dynamic CI, and selector-miss learning.

II. PUBLIC CAPABILITY BASELINE

GitHub Copilot cloud agent can work in the background, research a repository, create implementation plans, fix bugs, improve tests, update documentation, resolve merge conflicts, and operate in a GitHub Actions-powered environment [1]. Public limitations matter for control-plane design: the documented cloud agent operates on one branch at a time, opens one pull request per task, and is scoped to GitHub-hosted repositories [1]. GitHub exposes integrations through issues, pull requests, agents panels, IDEs, GitHub CLI, Slack, Teams, Linear, Jira, Azure Boards, and MCP-compatible tools [8], [10]. These are strong product surfaces, but they are not a local GitLab runner authority layer.

GitLab Duo Agent Platform, generally available in GitLab 18.8, positions agents across the software lifecycle with organization context and guardrails [2], [11]. GitLab describes Agentic Chat and Duo Workflow use cases for code analysis, CI/CD troubleshooting, planning, security analysis, issues, merge requests, and pipelines [3], [4], [12], [13]. GitLab’s public material also emphasizes human governance, enterprise context, and platform integration [14]. GITWRATH is complementary but narrower: it is not a general assistant; it is a GitLab-compatible control plane built to decide what an agent may do, what proof it must provide, and which local resources it may touch.

OpenHands is an open platform for software development agents that can write code, use terminals, and browse the web [5], [15], [16]. SWE-agent introduced an agent-computer interface for automated software engineering [6]. Aider provides

terminal-centered pair programming [7]. Codex-style local and cloud agents emphasize task execution across terminal, IDE, web, and pull request workflows [17]–[19]. These tools are valuable authoring engines, yet authoring is not the same as production authority. The control-plane question is: what validates the agent, records its authority, constrains its side effects, and prevents unsafe merge or release?

III. EMPIRICAL PRESSURE

The need for proof-carrying delivery follows from observed agent failure modes. Studies and datasets around software engineering agents show both promise and fragility: benchmark tasks can be solved, but real pull requests still require human modification, review, and rejection handling [20]–[23]. Work on agentic pull requests reports failed and unmerged agent contributions as a recurring operational pattern [24], [25]. Traceability research argues that software agents need stronger links among task, change, evidence, and outcome [26]. This is consistent with decades of automated repair work: a generated patch is not a proof of correctness [27].

Continuous integration already encodes a social contract: code is acceptable only after running the appropriate validation under a known configuration [28]. Regression test selection and prioritization literature adds nuance: selective testing can reduce cost, but unsafe selection can miss failures [29]–[31]. Agent-first systems stress this contract because they can produce more branches, more experiments, and more merge requests than humans can manually supervise. GITWRATH therefore treats validation choice itself as an auditable artifact.

IV. WHY GIT MUST CHANGE

Agent-first and “vibe coding” workflows shift the economic center of software delivery. When code generation becomes cheap, validation, review, provenance, and release authority become the scarce resources. The correct response is not to run fewer tests; it is to run the right tests, preserve proof for skipped tests, and make validation scale with the number of agent hypotheses. Coverage pressure will rise because agents can generate more code paths, more branches, and more plausible-but-wrong fixes than conventional human review can absorb.

Legacy Git forges were built around human-paced pull requests, human-scoped credentials, and CI as a background reaction to commits. That model can be extended with assistants, but extension is not the same as redesign. Agent-first development needs Git to understand task identity, capability grants, validation receipts, cache trust, runner isolation, proof-carrying merge decisions, and cleanup of speculative work. A Rust control plane is a pragmatic fit for this layer: it can sit close to Git, CI, filesystems, sockets, runners, and local policy while remaining fast enough for high-churn agent loops.

The paper’s central claim is therefore structural. Future developers will not keep pace with AI by waiting for hosted legacy platforms to retrofit every proof boundary. They need an open-source, agent-first substrate where validation and authority are first-class product concepts rather than conventions

Intent	agent action, actor, task, grant, idempotency key
Policy	path scope, branch scope, risk tier, trust tier, budget
Plan	VTI lanes, selected tests, skipped tests, fallback reason
Execute	runner pool, sandbox, cache namespace, CI pipeline
Evidence	logs, failure capsule, test receipt, cache verdict
Decide	merge gate, release gate, cleanup, durable memory

Fig. 1. GITWRATH proof loop. Agents do not jump directly from patch to merge; they move through intent, policy, validation, evidence, and decision records.

around a pull request. GITWRATH keeps GitLab compatibility where useful, but places the agent proof loop under local, inspectable control.

V. ARCHITECTURE

GITWRATH is a Rust workspace with a `vgit` binary and proof-scoped support crates. The control plane includes CLI and TUI surfaces, a Unix-socket capability API, GitLab webhook ingestion, Postgres-primary state with SQLite fallback, runner pools, custom executor hooks, VTI, release gates, and proof-routing utilities. It consumes GitLab concepts such as pipelines, runners, custom executors, protected branches, and CI artifacts [32]–[35]. It also interoperates conceptually with GitHub-style workflow constraints and branch protection semantics [36], [37].

The architecture differs from a bot wrapper. A bot wrapper receives an issue and emits a branch. GITWRATH owns the delivery loop around that branch. It can select a runner pool, inspect job state, retrieve failure capsules, reason about cache taint, schedule validation, and expose blockers. Postgres gives concurrent local agents a durable coordination layer, while SQLite remains the low-friction embedded fallback for single-node development and tests [38], [39]. Tokio and Axum support asynchronous API and webhook execution [40], [41]. Bollard gives Docker control for runner and container state [42]. Cargo-nextest, cargo-deny, RustSec, SLSA, in-toto, and Sigstore represent the surrounding proof and supply-chain ecosystem that GITWRATH is designed to coordinate with rather than replace [43]–[48].

VI. PROOF-SCOPED WORKSPACE

V3.01 treats repository structure as a proof-routing surface. The workspace contains support crates for witness graphs, validation routing, architectural exception review, runtime witness macros, and ARC/VRC benchmarking. These tools are intentionally local: they reduce the time an agent spends rediscovering ownership and validation rules. `cargo-witness` indexes public API signatures. `cargo-vrc` maps changed files to validation rings. `cargo-aer` audits structural exceptions. `witness-rt` emits structured repair packets when runtime expectations fail. `arc-bench` evaluates design tradeoffs.

TABLE I
VTI 3.0.1 VALIDATION OUTCOMES

Outcome	Meaning	Merge Semantics
passed	executed and passed	evidence-positive
failed	executed and failed	blocker
skipped_by_vti	skipped with receipt	evidence-qualified
docs_only	docs allowlist only	low-risk receipt
full_fallback	widened to full	conservative
unsafe_to_skip	no skip proof	must execute

This matters because an agent often needs two kinds of knowledge. The first is semantic knowledge about code. The second is procedural knowledge about how to prove a change. Public assistants focus heavily on the former. GITWRATH makes the latter explicit. A change under `src/secrets.rs` is not merely a Rust edit; it is a security-relevant proof lane. A change under `src/test_intel/` is not merely a module edit; it can affect selector correctness and should widen validation. This repository-level proof routing complements CI systems rather than duplicating them [32], [36].

VII. VTI: VALIDATION TEST INTELLIGENCE

VTI is the core latency reducer. The current internal planner maps changed paths to subsystems, applies global invalidators, identifies docs-only changes, selects unit and integration tests, and falls back to full validation when paths are unmapped or confidence is low. External repositories can provide `.vgit/testmap.toml`. V3.01 makes the conservative boundary sharper: unknown files force full validation; recursive subsystem patterns cover nested modules; selector misses become scoped evidence; and each skipped test must have a reason that a merge gate can inspect.

Selective validation is valuable only when the skipped work is explained. Classical regression test selection and prioritization show that reduction must be balanced against fault-detection risk [29]–[31]. GITWRATH therefore represents VTI as a proof generator, not an optimization flag. A VTI plan should say: these files changed; these subsystems are affected; these tests were selected; these tests were skipped; this skip is justified by path ownership, dependency information, and historical evidence; this unknown path caused full fallback.

The V3.01 code change moves this direction: unknown file types no longer collapse to docs-only behavior, nested TUI, gateway, and test-intelligence paths now match recursive subsystem patterns, internal plans can emit stable VTI receipts, and the default merge gate treats a missing receipt as a blocker. Remaining work is to persist receipts as first-class state, connect selector-miss records to oracle runs, and bind receipts to live GitLab job evidence. This distinction is essential. A skipped test with a receipt is a qualified claim. A missing test without a receipt is a hole.

VIII. CAPABILITY AND ADMISSION MODEL

The current capability API exposes useful intents: `RunTests`, `FetchCapsule`, `ProposePatch`, `RacePatches`, `RequestMerge`, `ExplainBlockers`,

TABLE II
MERGE GATE INPUTS

Input	Required Property
MR identity	project, IID, source, target, head SHA
Pipeline	latest for head SHA, terminal, required jobs passed
VTI	selected lanes, fallback status, skip receipts
Risk	security/release lanes, trust tier, approvals
Cache	no tainted artifacts in proof path
Branch	protected target, no conflict, no stale base
Evidence	capsules, logs, validation receipts retained

`GetSystemSnapshot`, `ListAllowedActions`, and `PlanValidation`. V3.01 adds an agent action envelope with request id, actor, nonce, expiry, optional budget, optional grant proof, and framed JSON transport while preserving legacy `AgentIntent` clients. The action registry remains canonical so API, CLI, and TUI do not drift. That is a small implementation change with a large governance consequence: an agent asking what it can do receives the same truth the human console uses.

The target authority model is an intent ledger. An agent requests an action with actor, task, grant, request, and idempotency identifiers. Policy evaluates scope, path rules, branch rules, TTL, risk tier, and budget. The ledger records allow, deny, or escalate before side effects occur. Evidence and cleanup records are attached afterward. This model aligns with the traceability requirement for agent contributions [26] and with supply-chain systems that separate declaration, execution, and attestation [46]–[48].

Admission control is the server-side counterpart. A pre-receive hook should not merely log suspicious input; it should reject policy violations with structured reasons. Examples include unscoped agent branches, forbidden paths, missing task IDs, unsigned privileged operations, stale base SHAs, or validation receipts that do not match the pushed commit. GitLab protected branch controls provide one perimeter [35]; GITWRATH adds agent-aware intent checks inside that perimeter.

IX. PROOF-CARRYING MERGE GATES

Autonomous merge requires stronger evidence than “some tests passed.” V3.01 introduces a shared merge gate used by CLI, TUI, capability API, and webhooks. The gate checks MR identity, exact head SHA, latest pipeline, required jobs, pending jobs, VTI plan confidence, selector misses, cache taints, security/release lanes, branch protection, approvals, conflicts, and stale branch state. Its output is a durable merge proof. A merge is allowed because evidence verifies, not because the agent is trusted.

The important design choice is that merge decisions are not embedded in one endpoint. A CLI command, TUI action, webhook reconciler, and capability request should call the same gate. Otherwise the system accumulates hidden bypasses. This is especially relevant for agents that can race patches or propose multiple branches. A patch-race feature is useful only if losing branches are cleaned up, winning branches still pass

the same gate, and the race result does not become a shortcut around required validation.

X. RUNNER, CACHE, AND SANDBOX BOUNDARIES

Agent-first development increases load on runners and caches. GITWRATH therefore treats execution context as part of proof. A trusted branch, an untrusted MR, a release job, and a supply-chain detonation lane should not share the same assumptions. GitLab Runner and custom executor documentation define the execution substrate [33], [34]; GITWRATH adds trust tiers, BuildKit namespace separation, taint tracking, honeypot detection, and cache promotion rules.

The cache problem is subtle. Smart caching makes CI faster, but unsafe cache reuse can smuggle state across trust boundaries. GITWRATH models cache decisions using trust tier, taint, and epoch. A cache hit is not merely a performance event; it is a statement that the artifact is acceptable for this build unit, namespace, and epoch. This aligns with broader supply-chain verification systems that treat artifacts, provenance, and signatures as first-class data [45]–[48]. Sandbox mechanisms such as gVisor, Firecracker, and Bubblewrap illustrate the isolation vocabulary available for untrusted work [49]–[51].

XI. AGENT WORKFLOWS

The intended V3.01 workflow starts with an explicit task. An agent receives a task description and requests capability grants. The control plane creates or verifies a branch, associates it with actor and task identity, and records an intent. The agent may then propose a patch, ask for validation, fetch failure capsules, race alternative hypotheses, or request a merge decision. Each operation is scoped by the same registry and policy layer, so the system can answer what was allowed, what was denied, and why.

A repair loop illustrates the difference from a generic coding assistant. The agent asks for a failing job trace. GITWRATH returns a capsule that includes project, job, pipeline, stage, log tail, suspected owner area, and suggested validation lanes. The agent patches the owner area and asks VTI for a plan. VTI selects tests or widens to full validation. The runner executes with the appropriate trust and cache namespace. The merge gate then checks the exact head SHA against the evidence. At no point does the system rely on the agent’s verbal claim that it ran tests.

Patch racing is the aggressive version of the same model. Multiple hypothesis branches can be launched in parallel, but every branch needs identity, cleanup policy, and evidence boundaries. The winner is not the fastest branch; it is the branch whose evidence satisfies the gate. Losers should be closed, caches should remain scoped, and the audit ledger should retain enough context to explain the decision. This is where an agent-first GitLab fork can exceed ordinary assistant integration: speed comes from orchestration, not from bypassing proof.

TABLE III
AGENT WORKFLOW STATE MACHINE

State	Agent Action	Control-Plane Proof
planned	request grant	task, actor, scope
patched	propose diff	branch, base SHA, paths
validated	run VTI plan	jobs, receipts, capsules
blocked	explain blockers	failing proof nodes
mergeable	request merge	gate decision
closed	cleanup	branch/cache records

XII. THREAT MODEL

GITWRATH assumes agents are useful but not intrinsically trusted. A capable agent may hallucinate validation, choose an unsafe shortcut, overfit a patch to a narrow test, leak a secret through logs, or unknowingly reuse tainted artifacts. A malicious or compromised agent may try to push to protected paths, create a branch that looks human-owned, replay old validation evidence, or exploit cache state. The defense is not one gate; it is layered evidence.

The first layer is identity and intent. Every privileged action needs actor, task, and idempotency context. The second layer is policy. Path scope, branch rules, risk tier, and grant TTL determine whether an action can proceed. The third layer is execution isolation. Runner pools, sandbox mechanisms, and cache namespaces reduce cross-contamination. The fourth layer is validation. VTI can reduce work only when it emits receipts; otherwise it must fall back. The fifth layer is final decision. Merge and release gates verify current head state, required jobs, approvals, taints, and blockers.

This model deliberately separates productivity from authority. An agent may be excellent at producing patches and still be untrusted for merge. A cache may be effective and still be unusable across trust tiers. A test plan may be fast and still be insufficient for a security-relevant path. V3.01’s central hardening target is to make those distinctions concrete in code rather than relying on convention.

XIII. TUI AND SCREENSHOTS

The TUI is a Ratatui operations surface, not a decorative dashboard [52]. It contains Mission, Release, Jobs, Agents, Tests, Pools, Cache, Evidence, and Secrets tabs. The V3.01 TUI upgrade applies the strongest review theme from the upgrade notes: the console should continuously answer what happened, why it matters, what proof exists, and what the safest next action is. The default Mission view now opens as an action cockpit with a Top Signal, Attention Queue, Proof Stack, next actions, metric tiles, and compact activity graphics. The Agents view now acts as an Agent Cockpit with phase, progress, branch/SHA, grants, timeline, and action guidance. The command palette previews risk, grant requirements, side effects, dry-run availability, and disabled reasons from the canonical action registry.

To support publication and review, V3.01 uses two capture modes:

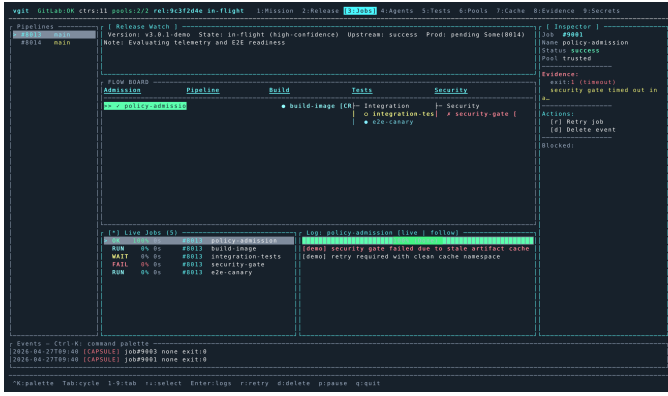


Fig. 2. TUI Jobs/Flow capture showing pipeline flow, live jobs, logs, and events.

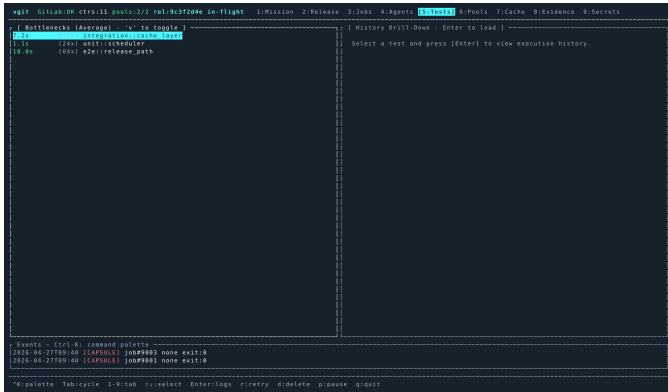


Fig. 3. TUI Tests/VTI capture showing selected validation, recent test intelligence, and history context.

- **PTY render:** runs the TUI inside a real pseudo-terminal, parses the screen state, and renders the grid with a pinned font and palette.
- **Deterministic render:** renders Ratatui TestBackend buffers for stable fixture diffs.

```
./scripts/capture-tui-screenshots.sh
just tui-screenshot-smoke
vgit tui --capture --tab jobs \
  --output paper/assets/vgit-tui-jobs-flow.png
```

The publication pipeline uses `vgit tui --screenshot` inside a real PTY and captures the terminal screen before alternate-screen cleanup. A dedicated `tui-capture` utility parses the terminal byte stream with a VT parser, validates that the selected font supports box drawing and block glyphs, and renders PNGs with fixed geometry, DejaVu Sans Mono, a lifted dark background, and brightened colors. This avoids browser, SVG, ANSI-to-image, and window-manager font fallback artifacts. The deterministic path remains useful for CI reproducibility. In both modes, screenshots are evidence artifacts backed by demo state snapshots.

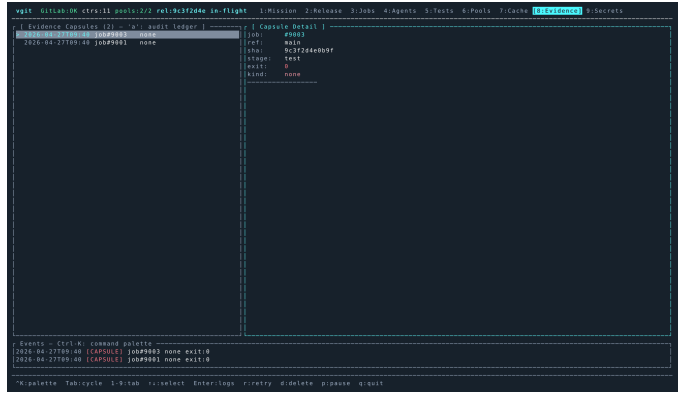


Fig. 4. TUI Evidence capture showing the audit/capsule surface that should become the human-readable proof ledger.

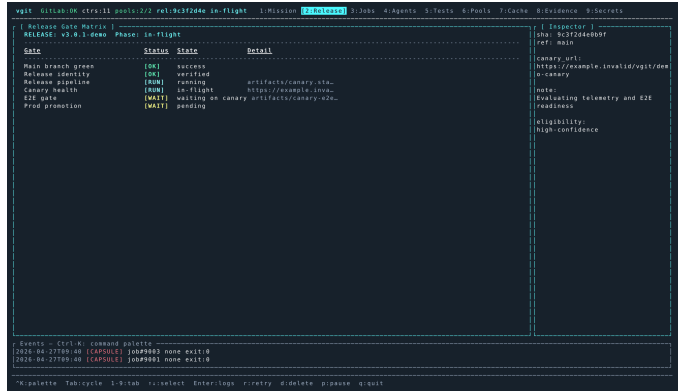


Fig. 5. TUI Release capture showing the release gate matrix and rollout status surfaces.

XIV. PAPER AND ARTIFACT STRATEGY

The V3.01 paper is both a publication artifact and an agent-readable specification. The LaTeX version targets IEEE format. The Markdown version mirrors the same argument for agents that need fast retrieval, patch planning, or review. The bibliography intentionally spans product documentation, open-source agents, empirical studies, regression test selection, CI, runner systems, supply-chain security, and sandboxing [1], [2], [5], [6], [20], [28], [29], [46], [49]. This range matters because GITWRATH is not one feature; it is an integration layer over several research and engineering traditions.

Generated screenshots are also artifacts. They prove that the paper is connected to the codebase. In V3.01 the screenshot command should become stable enough for CI to regenerate figures and fail if rendering changes unexpectedly. That turns visual drift into a reviewable change. For an agent-first project, this is useful: an agent can cite a screenshot, update it deterministically, and make the UI consequence of a patch explicit.

XV. COMPARISON TO EXISTING SURFACES

The clearest contrast is the boundary of authority. Public GitHub and GitLab agent surfaces expose high-level collaboration and repository assistance [1], [3], [4], [8]. OpenHands,

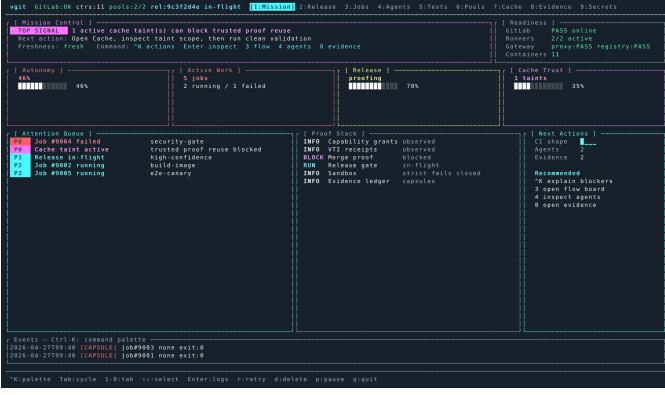


Fig. 6. TUI Mission capture showing action-first Mission Control with top signal, readiness, proof stack, attention queue, and next actions.

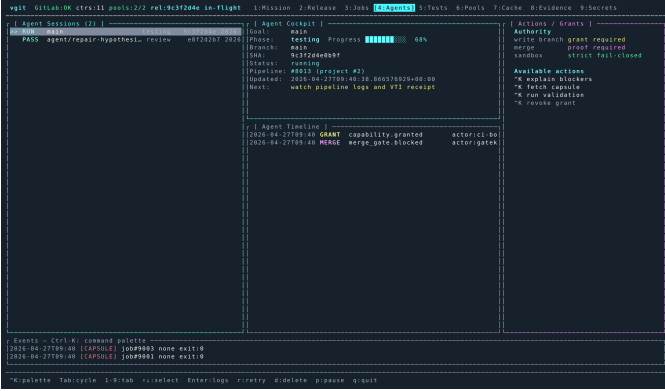


Fig. 7. TUI Agents capture showing the Agent Cockpit with session phase, progress, branch/SHA, grants, timeline, and action guidance.

SWE-agent, Aider, and Codex-style systems expose strong authoring and task execution capabilities [6], [7], [16], [17]. GITWRATH instead focuses on proof-scoped delivery authority. Its value is not that it can edit code; it assumes agents can already edit code. Its value is that it can constrain and verify the consequences of those edits inside a GitLab-compatible runtime.

XVI. PUBLIC PAIN POINTS

The public record already shows pressure around the exact boundaries GITWRATH targets. GitHub documents cloud-agent work as a GitHub-centered pull-request task flow, including constraints around assigned tasks and pull-request creation [1], [8]. Community reports around GitHub Actions describe queued workflows and runner availability frustration [53]. GitLab support material documents excessive queuing for pending CI/CD jobs with Docker Autoscaler runners, runner long-polling delays that can leave workers idle, and external status checks that remain pending and block merge requests [54]–[56]. GitLab’s own Duo Workflow design notes that ordinary CI pipelines are not a complete runtime substrate for agentic workflows [14].

These examples are not isolated product complaints; they are symptoms of a deeper mismatch. Agents need low-latency

TABLE IV
CAPABILITY CONTRAST

Surface	Public Focus	GITWRATH Focus
GitHub agent	PR task execution	local proof gates
GitLab Duo	platform assistance	runner authority
Open agents	coding autonomy	validation control
Aider/Codex	terminal/edit loop	merge/release proof
CI systems	job execution	evidence semantics

execution, branch/task identity, proof-aware validation, and authoritative merge/release decisions. A hosted forge can add agent features, but it still has to protect broad multi-tenant defaults. GITWRATH takes the opposite route: local control first, then agent velocity. The result is a sharper boundary for teams that want agents to move quickly without treating queued CI, stale checks, missing evidence, or untrusted cache reuse as ordinary inconvenience.

This distinction should not be read as a product criticism. Public systems optimize for broad adoption, hosted experience, and developer accessibility. GITWRATH optimizes for a narrower but harder case: agents that need to move fast inside a self-controlled GitLab environment without losing auditability. The implementation should therefore favor explicit local contracts over implicit cloud workflow assumptions.

XVII. IMPLEMENTATION STATUS

The current V3.01 tranche implements version tracking, conservative VTI fallback, recursive subsystem matching, canonical capability action listing, a V3 capability request envelope, nonce and expiry checks, action side-effect metadata, typed dynamic CI YAML generation, deterministic TUI PNG capture, high-fidelity screenshot capture, an action-first Mission Control view, an Agent Cockpit view, command-palette blast-radius previews, durable branch-write capability grants with commit-SHA binding for GitLab commit API writes, persisted admission decisions, ledger-aware admission enforcement for agent refs, VTI receipt generation, default merge-gate receipt enforcement, strict-sandbox fail-closed behavior, secure-by-default GitLab TLS, webhook push-event alignment, real active-agent issue listing, a Postgres-primary state backend with SQLite fallback, documentation updates, and paper assets. It also preserves the larger plan rather than pretending the system is complete. Known gaps are generated capability schemas, signed grant verification, peer-credential actor binding, path-scoped grant enforcement, SHA binding for non-commit Git write paths, complete GitLab MR/pipeline/approval merge evidence, selector-miss learning, richer event streaming, dedicated agent-run lifecycle state, and stronger tests around merge and release proof state.

XVIII. REPRODUCIBILITY

The repository should let a reviewer reproduce both code and paper artifacts. The minimum proof bundle for this draft is: workspace version 3.0.1; generated screenshots under `paper/assets`; `paper/main.tex`;

TABLE V
V3.01 WORK BREAKDOWN

Area	Current State	Next Proof
Versioning	3.0.1 tracked	release tags
VTI	receipts emitted	state persistence
Actions	registry-backed contract	schema tests
TUI	Mission/Agent cockpit + PNG capture	event stream
Merge	receipt-gated proof	GitLab evidence
Admission	ledger-aware hook	signed grants
CI YAML	typed emitter	template schemas

`paper/paper.md`; `paper/references.bib`; vendored IEEE class and bibliography style; `cargo check --workspace`; targeted VTI, impact, and TUI tests; `cargo-witness build`; and `cargo-vrc plan`. A future release candidate should add signed release tags, stored VTI proof receipts, merge-gate fixtures, admission-hook fixtures, and CI regeneration of fixtures.

Reproducibility also applies to negative results. If VTI widens to full validation because a file is unknown, that fact should be recorded. If a merge gate blocks because a pipeline is stale, that blocker should be durable. If a screenshot changes because the TUI changed, the diff should be reviewed. Agent-first development needs more than green checks; it needs a memory of why the checks were considered sufficient.

XIX. IMPLEMENTATION CONTRACTS

The most important V3.01 engineering work is to turn implicit behavior into contracts. A contract is stronger than a helper function. It has inputs, outputs, persisted evidence, fixtures, and a caller-independent interpretation. The first contract is the capability contract. Every action should expose a stable identifier, risk tier, surface list, dry-run availability, required grant, side-effect class, and schema for request and response. The existing registry-backed action list is the start of this contract. The next step is generated schema and contract tests that fail when API documentation, TUI action palette, and capability output disagree.

The second contract is the validation contract. A VTI plan should have a stable identifier, changed paths, repository identity, base SHA, head SHA, selected subsystems, selected tests, skipped tests, fallback reasons, and confidence. A validation receipt should bind that plan to actual job execution. If an agent changes code after validation, the receipt must no longer satisfy the merge gate. If a test is skipped, the receipt must say why. If the reason is unknown, the contract requires full validation. This contract is what lets smart test skipping remain compatible with conservative delivery.

The third contract is the merge contract. A merge decision should not be a boolean returned by a single command. It should be a record with inputs, blockers, decision, policy version, evidence references, and a timestamp. That record should be reproducible from stored state. If a pipeline is not the latest for the MR head SHA, the blocker should name both SHAs. If required approvals are missing, the blocker should

TABLE VI
CONTRACT ARTIFACTS

Contract	Artifact	Primary Test
Capability	action schema	registry/API parity
Validation	VTI receipt	selector fixtures
Merge	gate decision	false-allow tests
Release	rollout proof	preflight fixtures
Audit	evidence record	replay tests

name the rule. If cache taint is present, the blocker should name the artifact or namespace. An agent can then repair the real blocker instead of guessing.

The fourth contract is the release contract. Release decisions are more sensitive than merge decisions because they connect code to production. A release proof should include version, commit, canary status, production status, secret handoff state, preflight checks, artifact availability, and rollback path. This is where GITWRATH’s GitLab focus matters. A hosted coding assistant can prepare a pull request, but a local release control plane can reason about runner state, Vault state, registry state, disk pressure, and release artifacts in the same decision.

The fifth contract is the audit contract. Every proof object should be inspectable from both machine and human surfaces. Machines need JSON schemas and stable IDs. Humans need TUI panes, screenshots, and concise blocker summaries. Agents need both. The audit contract is therefore not a passive log; it is the substrate for future repair. A blocked merge, failed release, selector miss, or denied capability should become a structured input for the next agent step.

XX. V3.01 ROADMAP

The V3.01 roadmap is intentionally staged. The first stage establishes truthfulness: version numbers, changelog entries, registry-backed action lists, conservative VTI fallback, recursive subsystem patterns, and deterministic screenshots. This stage is already useful because it removes misleading behavior. Unknown file changes no longer look like documentation changes. The capability API no longer advertises a hand-maintained list. The paper can now be compiled from repository artifacts.

The second stage is authority. Capability requests now carry actor, request, nonce, expiry, budget, and optional grant context, while branch-write grants remain durable. Admission hooks need to reject more violations by default rather than only observe them. Dynamic CI generation now uses a typed serializer for the initial scope allowlist; the next step is reusable template schemas for all generated pipelines. These are boring interfaces by design; agents should find them predictable.

The third stage is proof. VTI receipts, merge-gate decisions, and release-gate decisions need schemas and fixtures. A selector miss should become training data for the next plan. A merge denial should be reproducible from stored inputs. A release block should point to exact missing artifacts or failed preflights. When the system says “safe to merge,” it should mean that the stored proof can be rechecked.

TABLE VII
ROADMAP GATES

Stage	Exit Criterion	Blocking Proof
Truth	contracts match code	check, unit
Authority	grants enforced	hook fixtures
Proof	gates persist receipts	integration
Operations	TUI/API parity	snapshots
Release	tag candidate	security lanes

The fourth stage is operational maturity. The TUI should gain panes for agent runs, merge gates, VTI plan details, patch races, and event-stream health. The capability API should expose the same state in machine-readable form. CI should regenerate screenshots and paper outputs. Security lanes should exercise taint, sandbox, secret, honeypot, and BuildKit namespace invariants. At that point V3.01 becomes more than a version label; it becomes a reproducible proof-control release.

XXI. EVALUATION PLAN

The evaluation should report VTI selection latency, test-count reduction, selector-miss rate under oracle runs, unknown-file fallback coverage, merge-gate false-allow rate in fixture scenarios, capability contract conformance, admission hook rejection coverage, TUI render determinism, screenshot determinism, and patch-race cleanup success. A credible evaluation should include three modes:

- 1) *inner loop*: targeted code changes where VTI should reduce tests without missing failures;
- 2) *oracle loop*: scheduled full runs that detect selector misses and train or adjust path rules; and
- 3) *adversarial loop*: untrusted branches, tainted cache inputs, stale SHAs, missing approvals, and manipulated validation receipts.

The key metric is not only speedup. Speedup without a false-allow bound is unsafe. The paper should therefore report both saved validation time and the number of cases where the system widened to full validation. Conservative fallback is a feature when evidence is missing.

XXII. RELATED WORK

GITWRATH is adjacent to coding agents, CI systems, regression test selection, and supply-chain security. Coding-agent systems focus on task execution and patch generation [5]–[7], [17], [18]. SWE-bench and related datasets measure repair capability [20]–[22]. Studies of agentic pull requests emphasize the human review and modification burden that remains [23]–[25]. CI and regression test selection research explain why validation scope must be explicit and measurable [28]–[31]. Supply-chain systems provide the vocabulary for provenance, attestation, signing, and policy [45]–[48]. Sandboxing projects define execution isolation tradeoffs [49]–[51]. GITWRATH’s contribution is to connect these ideas at the GitLab-compatible control-plane boundary.

XXIII. LIMITATIONS

The current repository is not yet the completed V3.01 system. Known gaps include signed capability grant verification, peer-credential actor binding, path-scope enforcement, GitLab-backed merge-gate completeness, generic repo configuration, event streaming, and deeper VTI evidence graphs. Some public capability comparisons are necessarily based on documented surfaces rather than private product internals. The screenshot workflow now includes both deterministic and interactive-terminal render modes, with regenerated figures in CI planned for both reproducibility and visual fidelity checks. These limitations are the work program, not footnotes.

XXIV. CONCLUSION

GITWRATH’s thesis is that the next agent platform is not just a better coding model. It is a proof-carrying control plane around Git, CI, runners, cache, evidence, merge, and release. V3.01 makes that thesis concrete: truthful versioning, conservative validation, canonical capabilities, deterministic screenshots, and a paper that turns the vision into a testable engineering agenda. The goal is faster agent behavior without reducing accountability. The system should let agents drive, but only on roads whose rules are explicit, inspected, and enforced.

APPENDIX A ARCHITECTURE DIAGRAM

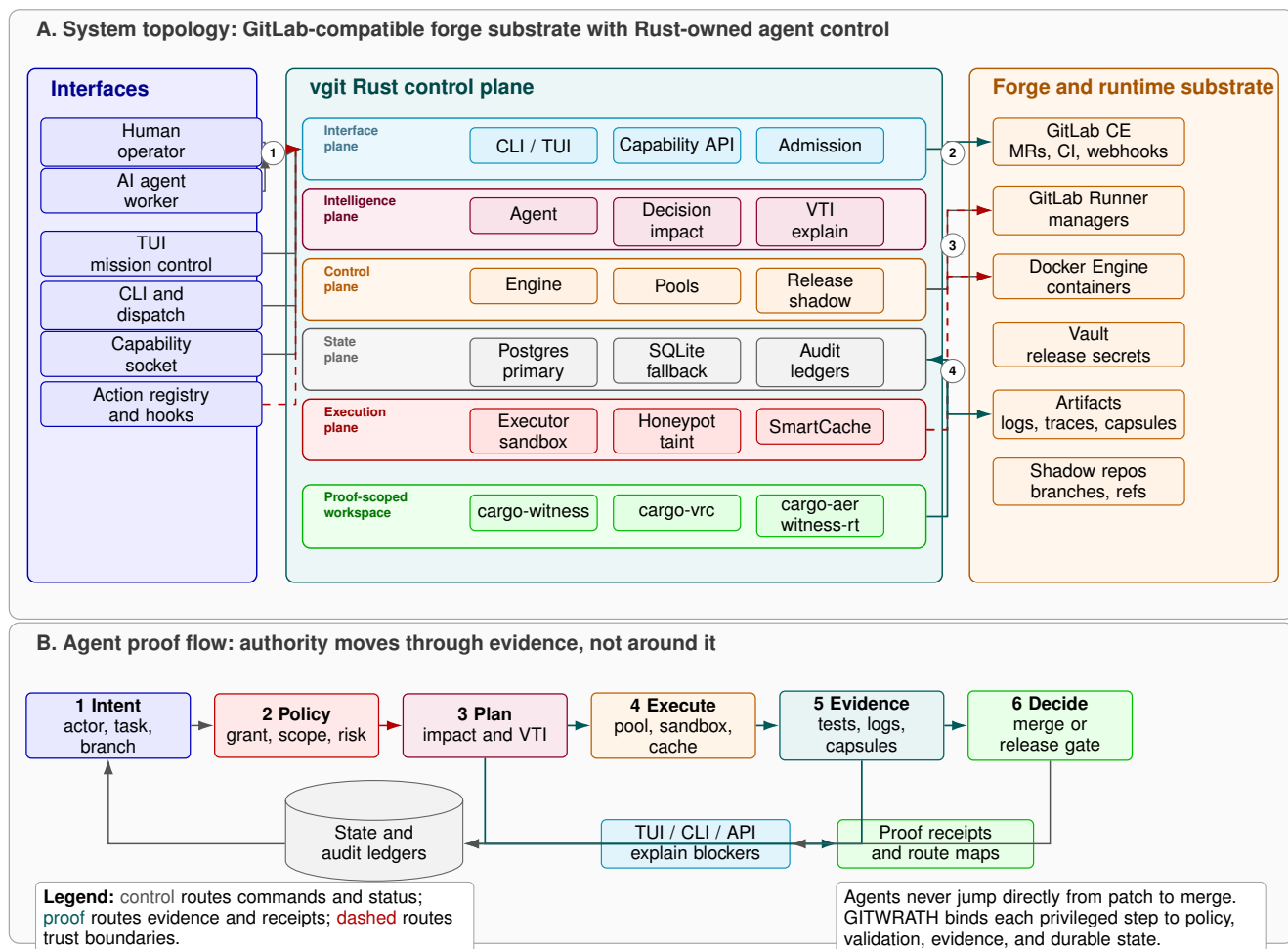


Fig. 8. GITWRATH architecture as described in `docs/ARCHITECTURE.md`. GitLab remains the forge and CI substrate, while the Rust `vgit` control plane owns agent authority, runner policy, validation planning, cache trust, evidence capture, merge gates, and release gates.

APPENDIX B

AGENT API AND CONTROL SURFACE REFERENCE

This appendix is the agent-facing contract for the V3.01 repository. It is grounded in the active `vgit` binary, the Unix-socket capability API, the native MCP server, the engine HTTP routes, the GitLab Runner custom executor protocol, the Git server admission hook, and the canonical action registry. The MCP surface is not a parallel authority system: it is a standards-compatible transport for the same policy, evidence, grant, and merge/release checks exposed through the existing capability model.

A. Surface Map

AGENT-RELEVANT API SURFACES				
Surface	Transport	Primary Consumer	Auth/Gate	Use
CLI	process invocation	humans, local agents, scripts	local process authority, GitLab PAT where needed	lifecycle, tests, release, pools, evidence, host operations
TUI	terminal UI	human supervisors, screen-reading agents	local process authority	operational state, actions, logs, proof stack
Capability API	Unix domain socket JSON	supervised local agents	protocol envelope, nonce, optional grant proof	typed agent intents and action discovery
MCP server	stdio JSON-RPC; loopback Streamable HTTP	MCP-compatible agents	local MCP lifecycle plus capability policy	standard tool discovery and calls over the same grant/evidence gates
HTTP engine	HTTP	GitLab webhooks, health checks	webhook token for /hooks	reconciliation, cache summary, webhooks
Server hook	stdin/stdout process hook	Git server / GitLab hook	admission ledger, optional enforce mode	pre-receive branch/ref policy
Custom executor	GitLab Runner protocol	runner managers	runner invocation contract	sandbox, cache, witness, failure capsule capture
Action registry	static JSON contract	CLI, TUI, capability clients	canonical risk/grant metadata	discover actions and required grants

Listing 1. Agent discovery and status endpoints

```
vgit action list --json
vgit status
vgit next --project-id 2 --ref-name main
vgit explain-blocker job 14445
vgit capability serve /tmp/vgit-capability.sock
vgit mcp serve
vgit mcp serve-http
vgit mcp tools --json
```

B. CLI Command Catalog

All command names are kebab-case at the shell. Defaults below match the current Clap definitions.

LIFECYCLE, TUI, STATUS, AND GLOBAL ACTION COMMANDS

Command	Mode	Purpose
<code>vgit init</code>	mutating	Bootstrap GitLab, state DB, runners, secrets, compose files, and smoke setup.
<code>vgit bootstrap</code>	mutating	Hidden alias for <code>init</code> .
<code>vgit serve</code>	daemon	Start Docker/GitLab orchestration, hook install, cache supervisor, runner warmup, engine, and shadow loop.
<code>vgit down</code>	mutating	Drain runner managers and stop the GitLab stack.
<code>vgit status</code>	read	Show GitLab, Vault, pools, managed containers, jobs, release, and secret state.
<code>vgit tui [--once]</code>	UI/read	Launch the Ratatui dashboard or render one smoke frame.
<code>vgit tui --capture --tab <tab> --output <png></code>	read/artifact	Deterministic Ratatui PNG capture. Tabs: mission, release, jobs, agents, tests, pools, cache, evidence, secrets.
<code>vgit tui --screenshot --tab <tab> --screenshot-hold-ms <ms></code>	read/session	Hold a real terminal render for external screenshot tooling.
<code>vgit progress --project-id 2 --ref-name main [--json]</code>	read	Show lane-aware CI and release progress.
<code>vgit next --project-id 2 --ref-name main</code>	read	Print the highest-priority next action for a branch.
<code>vgit explain-blocker <job release merge> <id></code>	read	Explain why an entity is blocked using DB/GitLab evidence.
<code>vgit action list [--json]</code>	read	Emit the canonical action registry for humans or agents.

POOL, JOB, PIPELINE, CACHE, AND HOST COMMANDS

Command	Mode	Purpose
<code>vgit pool list</code>	read	List pools, manager counts, paused state, executor, and runner id.
<code>vgit pool scale <name> <count></code>	mutating	Scale a pool to an exact manager count.
<code>vgit pool pause resume <name></code>	mutating	Pause or resume runner intake.
<code>vgit pool drain <name></code>	mutating	Pause, wait, and stop managers.
<code>vgit pool delete <name></code>	mutating	Drain, delete DB record, and unregister runner.
<code>vgit pool rotate-token <name></code>	mutating	Reset GitLab runner token and update manager config.
<code>vgit job list <project> [--status running,pending]</code>	read	List GitLab jobs by scope.
<code>vgit job trace <project> <job></code>	read	Print GitLab job trace.
<code>vgit job play cancel retry <project> <job></code>	mutating	Start manual jobs, cancel jobs, or retry failed jobs.
<code>vgit job explain <project> <job></code>	read	Print latest structured failure capsule and retry advice.
<code>vgit job clear</code>	mutating	Clear local job and pipeline history.
<code>vgit pipeline explain doctor jobs <pipeline> [--project-id 2] [--json]</code>	read	Inspect release eligibility, health, and job timing.
<code>vgit pipeline jobs <pipeline> --ingest</code>	mutating	Store current job timing rows.
<code>vgit pipeline ingest cancel <pipeline> [--project-id 2]</code>	mutating	Persist timings or cancel a pipeline.
<code>vgit pipeline bottlenecks [--limit 25] [--json]</code>	read	Report historical slow jobs from local timing ledger.
<code>vgit cache enable</code>	mutating	Configure Docker for SmartCache registry mirror.
<code>vgit cache doctor status [--json]</code>	read	Check proxy/registry/cache state and metrics.
<code>vgit cache gc [--dry-run] [--json] [--older-than <age>] [--max-cache-gb <gb>]</code>	mutating	Garbage-collect manager caches and cache state.
<code>vgit logs <manager-id> [-n 50]</code>	read	Tail manager container logs.
<code>vgit host storage-audit doctor [--json]</code>	read	Audit storage and host health.
<code>vgit host reclaim --mode <mode> (--plan --apply)</code>	mutating	Run planned or applied reclaim.

Command	Mode	Purpose
<code>vgit agent spawn <project> --task <text></code>	mutating	Create an autonomous task branch/MR/issue workflow.
<code>vgit agent list <project></code>	read	List active agent MRs/issues.
<code>vgit agent merge <project> <mr> [--trust-tier trusted]</code>	mutating	Request merge through the risk gate.
<code>vgit shadow <subcommand></code>	mixed	Manage periodic local source-to-GitLab shadow sync: add, enable, disable, remove, sync-now, status.
<code>vgit shadow-remote <subcommand></code>	mixed	Inspect, create, or push a repo-local shadow remote: status, ensure, push.
<code>vgit test run --command <cmd> [--force]</code>	mutating	Run a command through an ephemeral GitLab CI pipeline.
<code>vgit test plan --command <cmd></code>	read	Preview inferred image, tags, timeout, and runner class.
<code>vgit test batch --command <cmd>... [--max-parallel 3]</code>	mutating	Launch several validation pipelines concurrently.
<code>vgit test results failed retry <pipeline></code>	mixed	Inspect or retry pipeline test jobs.
<code>vgit test impact --base <sha> --head <sha> [--json]</code>	read	Determine CI jobs and release gates needed for a diff.
<code>vgit test select [emit flags]</code>	read/artifact	Compute internal VTI plan and optionally emit GitLab YAML, JSON plan, and VTI receipt.
<code>vgit test explain-plan <plan.json></code>	read	Explain a serialized VTI plan.
<code>vgit test select-external --workspace <root></code>	read/artifact	Use external <code>.vgit/testmap.toml</code> .
<code>vgit test audit learn</code>	read	Compare VTI selections with oracle results and suggest rule changes.
<code>vgit test cache-status --base <sha> --head <sha> [--json]</code>	read	Show test-command cache state for the current source state.
<code>vgit release status watch reconcile</code>	mixed	Read or reconcile release attempts.
<code>vgit release promote-prod [--version <v>]</code>	mutating	Trigger approved production promotion after canary gates.
<code>vgit release preflight doctor [--json]</code>	read	Check blockers, infrastructure, and release readiness.
<code>vgit secrets <subcommand></code>	mixed	Manage Vault-backed release lifecycle: init, status, rotate, finalize, report, recover.
<code>vgit repo render-agent-index [--check]</code>	artifact/check	Generate or verify machine-readable agent routing index.
<code>vgit repo audit-agent-surface [--json]</code>	read	Audit routing docs, generated index, and agent surface freshness.

C. Exact CLI Grammar

The following grammar is generated from the active Clap definitions in `src/cli.rs`. It includes hidden protocol calls because those are part of the agent and runner contract even though they are not intended for direct human operation.

Listing 2. Exhaustive vgit command forms

```

vgit init
vgit bootstrap
vgit serve
vgit down
vgit status
vgit logs <manager-id> [-n <lines>]
vgit progress [--project-id <id>] [--ref-name <ref>] [--json]
vgit next [--project-id <id>] [--ref-name <ref>]
vgit explain-blocker <job|release|merge> <id>

vgit tui [--once]
vgit tui --capture [--tab <tab>] [--output <path>] [--width <cols>] [--height <rows>]
vgit tui --screenshot [--tab <tab>] [--width <cols>] [--height <rows>] [--screenshot-hold-ms <ms>]

vgit action list [--json]

vgit pool list
vgit pool scale <name> <count>
vgit pool pause <name>

```

```

vgit pool resume <name>
vgit pool drain <name>
vgit pool delete <name>
vgit pool rotate-token <name>

vgit job list <project-id> [--status <csv>]
vgit job trace <project-id> <job-id>
vgit job play <project-id> <job-id>
vgit job cancel <project-id> <job-id>
vgit job retry <project-id> <job-id>
vgit job explain <project-id> <job-id>
vgit job clear

vgit pipeline explain [--project-id <id>] <pipeline-id> [--json]
vgit pipeline doctor [--project-id <id>] <pipeline-id> [--json]
vgit pipeline jobs [--project-id <id>] <pipeline-id> [--ingest] [--json]
vgit pipeline ingest [--project-id <id>] <pipeline-id>
vgit pipeline cancel [--project-id <id>] <pipeline-id>
vgit pipeline bottlenecks [--project-id <id>] [--ref-name <ref>] [--limit <n>] [--json]

vgit cache enable
vgit cache doctor
vgit cache status [--json]
vgit cache gc [--dry-run] [--json] [--keep-active-managers[=true|false]] [--older-than <age>] [--max-cache-gb <gb>]

vgit agent spawn <project-id> --task <text>
vgit agent list <project-id>
vgit agent merge <project-id> <mr-iid> [--trust-tier <tier>]

vgit shadow add --source <path> --project-id <id> --branch <branch> [--enable]
vgit shadow enable --source <path>
vgit shadow disable --source <path>
vgit shadow remove --source <path>
vgit shadow sync-now --source <path>
vgit shadow status [--source <path>]

vgit shadow-remote status [--repo <path>] [--name <remote>]
vgit shadow-remote ensure [--repo <path>] [--name <remote>] --url <url>
vgit shadow-remote push [--repo <path>] [--name <remote>] [--branch <branch>] [--mirror]

vgit test run --command <cmd> [--project-id <id>] [--image <image>] [--tags <csv>] [--timeout <sec>] [--force]
vgit test plan --command <cmd> [--project-id <id>] [--image <image>] [--tags <csv>] [--timeout <sec>]
vgit test batch --command <cmd>... [--project-id <id>] [--image <image>] [--tags <csv>] [--timeout <sec>] [--max-parallel <n>] [--force]
vgit test results <pipeline-id> [--project-id <id>]
vgit test retry <pipeline-id> <job-name> [--project-id <id>]
vgit test failed <pipeline-id> [--project-id <id>]
vgit test impact --base <ref> --head <ref> [--repo-root <path>] [--json]
vgit test select [--base <ref>] [--head <ref>] [--repo-root <path>] [--explain] [--json] [--emit-gitlab <path>] [--emit-plan <path>] [--emit-receipt <path>]
vgit test explain-plan <plan-path>
vgit test select --external [--base <ref>] [--head <ref>] --workspace <path> [--explain] [--json] [--emit-gitlab <path>] [--emit-plan <path>] [--emit-skipped <path>]
vgit test audit --changed <csv> --all-tests <csv> [--failed <csv>] [--sha <sha>] [--json] [--workspace <path>]
vgit test learn --changed <csv> --all-tests <csv> [--failed <csv>] [--sha <sha>] [--json] [--workspace <path>]
vgit test cache-status [--base <ref>] [--head <ref>] [--json]

vgit release status [--project-id <id>] [--ref-name <ref>] [--sha <sha>] [--limit <n>] [--json]
vgit release watch [--project-id <id>] [--ref-name <ref>] [--sha <sha>] [--limit <n>] [--interval-secs <sec>] [--json]
vgit release reconcile [--project-id <id>] [--ref-name <ref>] [--json]
vgit release promote-prod [--project-id <id>] [--ref-name <ref>] [--version <version>]
vgit release preflight [--ssh-host <host>] [--json]
vgit release doctor [--version <version>] [--preflight[=true|false]] [--json]

vgit secrets init
vgit secrets status [--json]
vgit secrets rotate [--repo <name>] --version <version> --target <target>
vgit secrets finalize [--repo <name>] --version <version> --target <target>
vgit secrets report [--repo <name>] --version <version>
vgit secrets recover [--repo <name>] --version <version>

vgit host storage-audit
vgit host doctor [--json]
vgit host reclaim --mode <mode> [--plan] [--apply]

vgit repo render-agent-index [--check]
vgit repo audit-agent-surface [--json]

vgit exec config
vgit exec prepare
vgit exec run <script-path> <stage>
vgit exec cleanup
vgit server-hook pre-receive

```

```
vgit capability serve <socket-path>
```

D. Hidden Protocol Entrypoints

PROTOCOL COMMANDS NOT INTENDED FOR DIRECT HUMAN USE

Command	Caller	Contract
vgit exec config	GitLab Runner	Return custom executor driver configuration JSON.
vgit exec prepare	GitLab Runner	Prepare clone/sandbox/honeypot/cache context.
vgit exec run <script> <stage>	GitLab Runner	Run a build stage with cache brain, witnesses, tripwire handling, and capsule capture.
vgit exec cleanup	GitLab Runner	Clean executor state after job completion.
vgit server-hook pre-receive	Git server hook	Read ref updates from stdin and write admission decisions.
vgit capability serve <socket>	local supervisor	Start Unix-socket JSON capability API.

E. Capability Socket Protocol

The capability API is a local Unix domain socket service. It accepts either a legacy tagged `AgentIntent` JSON document or the V3.01 `AgentActionRequest` envelope. New agents should use the envelope. The server also accepts an optional 4-byte big-endian length prefix before the JSON body. Responses use `CapabilityResponse`.

Listing 3. V3.01 capability envelope

```
struct AgentActionRequest {
  protocol_version: "v3.01",
  request_id: String,
  actor: String,
  nonce: String,
  expires_at: Option<Rfc3339DateTime>,
  project_id: Option<i64>,
  base_ref: Option<String>,
  base_sha: Option<String>,
  idempotency_key: Option<String>,
  budget: Option<ActionBudget>,
  grant: Option<CapabilityGrantProof>,
  intent: AgentIntent
}

struct CapabilityResponse {
  success: bool,
  message: String,
  data: Option<JsonValue>
}
```

Listing 4. Envelope example: request a system snapshot

```
{
  "protocol_version": "v3.01",
  "request_id": "req-20260427-0001",
  "actor": "agent.codex.local",
  "nonce": "7f096ab2-9e0d-4d80-9c4b-42fb56c6d62d",
  "expires_at": "2026-04-27T10:15:00Z",
  "idempotency_key": "snapshot-main-001",
  "intent": { "intent": "GetSystemSnapshot" }
}
```

ENVELOPE VALIDATION AND REPLAY RULES

Field/Rule	Behavior
protocol_version	Must equal v3.01; other versions are rejected.
request_id and actor	Required for enveloped requests and logged as request context.
nonce	Required and cached per actor; duplicate actor/nonce pairs are rejected as replay.
expires_at	Optional RFC3339 timestamp; expired requests are rejected.
budget	Optional caller-declared time/branch/pipeline limits.
grant	Optional proof payload. Current code validates envelope shape and records grants for successful branch-writing intents; signed grant enforcement remains future hardening.
Legacy requests	Accepted for compatibility, tagged as <code>legacy-capability-client</code> .

F. Capability Intents

CAPABILITY INTENTS AND EFFECTS

Intent	Risk	Inputs	Result
FetchCapsule	read	job_id	Return latest DB failure capsule for a job, or a structured miss.
RunTests	CI execution	project_id, target_ref, test_scope	Create ephemeral branch, write dynamic CI YAML, trigger pipeline, record a 24-hour grant.
ProposePatch	git write	project, branch, base, commit message, file modifications, optional MR title	Create branch, commit changes, open MR, record grant bound to returned commit SHA.
RacePatches	git write / CI	project, base branch, commit message, hypotheses	Launch multiple hypothesis branches and pipelines; returns branches, pipeline ids, grant ids.
RequestMerge	production/merge	project, MR IID, source, target	Evaluate conservative merge gate proof using selector miss and cache taint checks.
ExplainBlockers	read	entity type and id	Explain blockers for job, release, or merge-like entity.
GetSystemSnapshot	read	none	Return pools, cache metrics, selector misses, recent failures, timestamp.
ListAllowedActions	read	none	Return capability-visible registry actions with risk/grant metadata.
PlanValidation	read	project, test ids, ref	Validate a proposed test list against recent selector misses.

Listing 5. Legacy intent examples

```
{ "intent": "FetchCapsule", "payload": { "job_id": 14445 } }

{
  "intent": "RunTests",
  "payload": {
    "project_id": 2,
    "target_ref": "main",
    "test_scope": "unit"
  }
}

{
  "intent": "PlanValidation",
  "payload": {
    "project_id": 2,
    "ref_name": "main",
    "test_ids": ["unit:vgit", "integration:job_tests"]
  }
}
```

Listing 6. ProposePatch payload shape

```
{
  "intent": "ProposePatch",
  "payload": {
    "project_id": 2,
    "branch_name": "agent/fix-cache-proof",
    "base_ref": "main",
    "commit_message": "fix: strengthen cache proof",
    "mr_title": "Agent patch: cache proof hardening",
    "modifications": [
      {
        "file_path": "src/cache_brain.rs",
        "content": "...complete replacement content..."
      }
    ]
  }
}
```

G. HTTP Engine API

ENGINE HTTP ROUTES

Method	Path	Auth	Behavior
GET	/health	none	Return ok.
POST	/hooks	X-Gitlab-Token matching VGIT_WEBHOOK_SECRET	Ingest GitLab job, pipeline, push, and MR webhooks.
GET	/cache/summary	none	Return cache health summary JSON.

Listing 7. HTTP route examples

```
curl http://127.0.0.1:9777/health
curl http://127.0.0.1:9777/cache/summary
curl -X POST http://127.0.0.1:9777/hooks \
  -H "X-Gitlab-Token: $VGIT_WEBHOOK_SECRET" \
  -H "X-Gitlab-Event: Push Hook" \
  --data-binary @push-hook.json
```

WEBHOOK EVENT BEHAVIOR

Event	Behavior
Job Hook	Upsert job event, maybe auto-retry failures, scale up for pending/created work.
Pipeline Hook	Track pipeline state, launch canary on green main, update release/prod pipeline state.
Push Hook	Normalize ref, skip vgit-test-*, cancel superseded pipelines, compute impact, persist VTI plan.
Merge Request Hook	Logged for observability; no privileged merge behavior.
Other	Logged as unhandled.

H. Action Registry Contract

The action registry is the shared action catalog consumed by the TUI command palette, `vgit action list`, and `ListAllowedActions`. Each entry exposes `id`, `label`, `key_hint`, `risk_tier`, `side_effect_class`, `required_grant`, `dry_run`, `surfaces`, and `description`.

CANONICAL ACTION REGISTRY

Action ID	Risk	Grant	Surfaces	Purpose
open_logs	read-only	none	TUI	Open selected job logs.
retry_job	low	agent task	CLI, TUI	Retry a failed or canceled job.
delete_record	low	agent task	TUI	Remove local DB record.
pause_pool	low	agent task	CLI, TUI	Pause or resume runner pool.
explain_blockers	read-only	none	CLI, TUI, capability	Explain job/release/merge blockers.
get_system_snapshot	read-only	none	CLI, capability	Return system summary.
propose_patch	high	agent task	CLI, TUI, capability	Create branch, patch, and MR.
race_patches	high	agent task	CLI, capability	Launch competing hypotheses.
request_merge	production	merge approval	CLI, TUI, capability	Request merge through gate.
plan_validation	read-only	none	CLI, capability	Validate proposed test plan.
run_tests	low	agent task	CLI, TUI, capability	Trigger targeted test pipeline.
next_action	read-only	none	CLI, TUI	Print recommended next action.
tab_*	read-only	none	TUI	Switch primary TUI tab from Mission through Secrets.
toggle_audit_ledger	read-only	none	TUI	Toggle Evidence tab view.
quit	read-only	none	TUI	Exit TUI.

Listing 8. Action registry JSON excerpt

```
{
  "id": "request_merge",
  "label": "Request merge",
  "risk_tier": "production",
  "side_effect_class": "merge",
  "required_grant": "merge_approval",
```

```
"dry_run": false,  
"surfaces": ["cli", "tui", "capability"],  
"description": "Merge an MR through the risk gate..."  
}
```

TABLE VIII
MCP SERVER TRANSPORTS AND LIFECYCLE

Surface	Behavior
stdio	JSON-RPC line protocol for local agents launched as child processes by editors, terminals, or agent runtimes.
HTTP	Streamable HTTP endpoint at <code>/mcp</code> , bound to loopback by default through <code>Settings.mcp.bind</code> .
initialize	Negotiates protocol version 2025-11-25, returns <code>ServerInfo</code> , and advertises tool support.
notifications/ initialized	Marks the session initialized and records client attribution for later capability context.
ping	Lightweight liveness check for clients that keep long-running sessions.
tools/list	Returns the canonical tool manifest derived from the action registry.
tools/call	Parses tool arguments, builds an <code>AgentIntent</code> , executes it through capability policy, and returns MCP content plus structured result data.

TABLE IX
MCP TOOL SURFACE

Tool	Function
<code>vgit.get_system_snapshot</code>	Read current operational state for jobs, pools, release, and evidence.
<code>vgit.explain_blockers</code>	Explain why a job, release, or merge is blocked.
<code>vgit.plan_validation</code>	Produce a VTI-backed validation plan for selected tests and refs.
<code>vgit.fetch_capsule</code>	Fetch structured failure capsule data for a job.
<code>vgit.run_tests</code>	Create an ephemeral validation branch and trigger CI for a scoped test run.
<code>vgit.propose_patch</code>	Create a branch, apply file modifications, commit, and open a merge request.
<code>vgit.race_patches</code>	Launch multiple patch hypotheses while preserving branch-level evidence.
<code>vgit.request_merge</code>	Ask the merge gate to evaluate an MR against release and risk policy.

I. Native MCP Server

V3.01 ships a native Model Context Protocol server for external coding agents that already speak MCP. The server is implemented as `src/mcp.rs` plus the `src/mcp/` module tree and is exposed through `vgit mcp serve`, `vgit mcp serve-http`, and `vgit mcp tools --json`. Its purpose is deliberately narrow: translate MCP lifecycle and tool messages into the same typed `AgentIntent` execution path used by the Unix-socket capability API.

The important boundary is that MCP is a transport, not a privilege escalation path. A call to `vgit.request_merge` over MCP is evaluated by the same capability policy, grant metadata, durable evidence recording, GitLab client wrapper, and merge/release gate logic as the equivalent native capability request. Read-only tools remain read-only; branch-writing tools still create explicit branches and evidence; production-sensitive tools still require the same gate decisions.

Tool discovery is schema-bearing. Each MCP tool includes an `inputSchema`, an `outputSchema`, and MCP annotations such as `readOnlyHint`, `destructiveHint`, `idempotentHint`, and `openWorldHint`. These annotations are derived from the same action-risk vocabulary used by the CLI and TUI registry, so an agent does not need a separate policy document to know

which tools are read-only, mutating, or production-sensitive.

The HTTP transport adds local-server hardening around the same core. It refuses non-loopback bind addresses at startup, rejects non-local `Origin` headers, returns `McpSession-Id` on `initialize`, requires that session id on later requests, requires `MCP-Protocol-Version` after initialization, validates `Mcp-Method` and `Mcp-Name` against the JSON-RPC body, and supports `DELETE /mcp` for explicit session teardown. `GET /mcp` returns 405; this implementation is a POST-oriented local control surface rather than an event-streaming API.

Listing 9. MCP tool call shape

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "vgit.explain_blockers",
    "arguments": {
      "entity_type": "merge",
      "entity_id": 42
    }
  }
}
```

The response shape is intentionally useful to both simple chat agents and structured orchestrators. `MCP content` carries a human-readable text summary, while `structuredContent` carries the native capability response with `success`, `message`, and optional data. When the underlying capability call fails, the server sets `isError` without hiding the structured policy result. This lets clients present a concise explanation to a human while still preserving machine-readable evidence for retries, escalation, or audit.

REFERENCES

- [1] GitHub, “About github copilot cloud agent,” <https://docs.github.com/en/copilot/concepts/agents/cloud-agent/about-cloud-agent>, 2026, accessed 2026-04-27.
- [2] GitLab, “Gitlab announces the general availability of gitlab duo agent platform,” <https://about.gitlab.com/press/releases/2026-01-15-gitlab-announces-duo-agent-platform-general-availability/>, 2026, accessed 2026-04-27.
- [3] —, “Gitlab duo agent platform,” <https://about.gitlab.com/gitlab-duo/agent-platform/>, 2026, accessed 2026-04-27.
- [4] —, “Gitlab duo workflow solution components,” https://docs.gitlab.com/solutions/components/duo_workflow/, 2026, accessed 2026-04-27.
- [5] X. Wang *et al.*, “Openhands: An open platform for ai software developers as generalist agents,” *arXiv preprint arXiv:2407.16741*, 2024.
- [6] J. Yang *et al.*, “Swe-agent: Agent-computer interfaces enable automated software engineering,” *arXiv preprint arXiv:2405.15793*, 2024.
- [7] P. Gauthier, “Aider: Ai pair programming in your terminal,” <https://github.com/Aider-AI/aider>, 2026, accessed 2026-04-27.
- [8] GitHub, “Github copilot cloud agent,” <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/cloud-agent>, 2026, accessed 2026-04-27.
- [9] GitLab, “Gitlab duo agent platform documentation,” https://docs.gitlab.com/user/duo_agent_platform/, 2026, accessed 2026-04-27.
- [10] Model Context Protocol, “Model context protocol specification,” <https://modelcontextprotocol.io/specification>, 2026, accessed 2026-04-27.
- [11] GitLab, “Gitlab 18.8 released with gitlab duo agent platform generally available,” <https://about.gitlab.com/releases/2026/01/15/gitlab-18-8-released/>, 2026, accessed 2026-04-27.

- [12] —, “Introduction to gitlab duo agent platform,” <https://about.gitlab.com/blog/introduction-to-gitlab-duo-agent-platform/>, 2026, accessed 2026-04-27.
- [13] —, “Gitlab duo workflow: Enterprise visibility and control for agentic ai,” <https://about.gitlab.com/blog/2025/02/24/gitlab-duo-workflow-enterprise-visibility-and-control-for-agentic-ai/>, 2025, accessed 2026-04-27.
- [14] —, “Gitlab duo workflow design document,” https://handbook.gitlab.com/handbook/engineering/architecture/design-documents/duo_workflow/, 2026, accessed 2026-04-27.
- [15] OpenHands, “Openhands: The open platform for cloud coding agents,” <https://openhands.dev/>, 2026, accessed 2026-04-27.
- [16] —, “Openhands: Ai-driven development,” <https://github.com/OpenHands/OpenHands>, 2026, accessed 2026-04-27.
- [17] OpenAI, “Codex: Ai coding partner from openai,” <https://openai.com/codex/>, 2026, accessed 2026-04-27.
- [18] GitHub, “Openai codex in github copilot,” <https://docs.github.com/copilot/concepts/agents/openai-codex>, 2026, accessed 2026-04-27.
- [19] OpenAI, “Addendum to o3 and o4-mini system card: Codex,” https://cdn.openai.com/pdf/8df7697b-c1b2-4222-be00-1fd3298f351d/codex_system_card.pdf, 2025, accessed 2026-04-27.
- [20] C. E. Jimenez *et al.*, “Swe-bench: Can language models resolve real-world github issues?” *International Conference on Learning Representations*, 2024.
- [21] OpenAI, “Swe-bench verified,” *Benchmark documentation*, 2024.
- [22] Unknown, “Aidev: Studying ai coding agents on github,” *arXiv preprint arXiv:2602.09185*, 2026.
- [23] —, “How ai coding agents modify code: A large-scale study of github pull requests,” *arXiv preprint arXiv:2601.17581*, 2026.
- [24] —, “Where do ai coding agents fail? an empirical study of failed agentic pull requests in github,” *arXiv preprint arXiv:2601.15195*, 2026.
- [25] —, “Why are ai agent involved pull requests remain unmerged? an empirical study,” *arXiv preprint arXiv:2602.00164*, 2026.
- [26] —, “Understanding software engineering agents through the lens of traceability: An empirical study,” *arXiv preprint arXiv:2506.08311*, 2025.
- [27] M. Monperrus, “Automatic software repair: A bibliography,” *ACM Computing Surveys*, 2018.
- [28] M. Fowler and M. Foemmel, “Continuous integration,” <https://martinfowler.com/articles/continuousIntegration.html>, 2006.
- [29] S. Yoo and M. Harman, “Regression testing minimization, selection and prioritization: A survey,” *Software Testing, Verification and Reliability*, 2012.
- [30] G. Rothermel and M. J. Harrold, “A safe, efficient regression test selection technique,” *ACM Transactions on Software Engineering and Methodology*, 1997.
- [31] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, “Prioritizing test cases for regression testing,” *IEEE Transactions on Software Engineering*, 2001.
- [32] GitLab, “Gitlab ci/cd documentation,” <https://docs.gitlab.com/ci/>, 2026, accessed 2026-04-27.
- [33] —, “Gitlab runner documentation,” <https://docs.gitlab.com/runner/>, 2026, accessed 2026-04-27.
- [34] —, “Gitlab runner custom executor,” <https://docs.gitlab.com/runner/executors/custom/>, 2026, accessed 2026-04-27.
- [35] —, “Protected branches,” <https://docs.gitlab.com/user/project/repository/branches/protected/>, 2026, accessed 2026-04-27.
- [36] GitHub, “Github actions documentation,” <https://docs.github.com/actions>, 2026, accessed 2026-04-27.
- [37] —, “About protected branches,” <https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>, 2026, accessed 2026-04-27.
- [38] PostgreSQL Global Development Group, “Postgresql documentation,” <https://www.postgresql.org/docs/>, 2026, accessed 2026-04-27.
- [39] SQLite, “Sqlite documentation,” <https://www.sqlite.org/docs.html>, 2026, accessed 2026-04-27.
- [40] Tokio, “Tokio asynchronous runtime documentation,” <https://tokio.rs/>, 2026, accessed 2026-04-27.
- [41] —, “Axum web framework,” <https://github.com/tokio-rs/axum>, 2026, accessed 2026-04-27.
- [42] Bollard contributors, “Bollard docker api client for rust,” <https://github.com/fussybeaver/bollard>, 2026, accessed 2026-04-27.
- [43] Nextest contributors, “cargo-nextest,” <https://nexte.st/>, 2026, accessed 2026-04-27.
- [44] Embark Studios, “cargo-deny,” <https://github.com/EmbarkStudios/cargo-deny>, 2026, accessed 2026-04-27.
- [45] RustSec, “Rustsec advisory database,” <https://rustsec.org/>, 2026, accessed 2026-04-27.
- [46] OpenSSF, “Supply-chain levels for software artifacts,” <https://slsa.dev/>, 2026, accessed 2026-04-27.
- [47] in-toto contributors, “in-toto: A framework to secure the integrity of software supply chains,” <https://in-toto.io/>, 2026, accessed 2026-04-27.
- [48] Sigstore, “Sigstore documentation,” <https://docs.sigstore.dev/>, 2026, accessed 2026-04-27.
- [49] Google, “gvisor documentation,” <https://gvisor.dev/docs/>, 2026, accessed 2026-04-27.
- [50] Firecracker, “Firecracker microvm,” <https://firecracker-microvm.github.io/>, 2026, accessed 2026-04-27.
- [51] Project Atomic, “Bubblewrap,” <https://github.com/containers/bubblewrap>, 2026, accessed 2026-04-27.
- [52] Ratatui, “Ratatui documentation,” <https://ratatui.rs/>, 2026, accessed 2026-04-27.
- [53] GitHub Community, “Github actions taking too long, stuck in queued for hours,” <https://github.com/orgs/community/discussions/165400>, 2026, accessed 2026-04-27.
- [54] GitLab Support, “Excessive queuing of pending ci/cd jobs with docker autoscaler runner,” <https://support.gitlab.com/hc/en-us/articles/21409216187292-Excessive-queuing-of-Pending-CI-CD-jobs-with-Docker-Autoscaler-Runner>, 2024, accessed 2026-04-27.
- [55] GitLab, “Troubleshooting gitlab runner,” <https://docs.gitlab.com/runner/faq/>, 2026, accessed 2026-04-27.
- [56] GitLab Support, “Troubleshooting external status checks that hang or stay pending,” <https://support.gitlab.com/hc/en-us/articles/23099475806748-Troubleshooting-External-Status-Checks-That-Hang-or-Stay-Pending>, 2024, accessed 2026-04-27.